2026-10-12 题解与思路分析汇总

Tangyixiao

2026-10-12

目录

P8378 [PFOI Round1] Two Sequences 题解 2

P8380 Two Hypercubes 题解 4

P8412 「SvR-1」Hack Function! 题解 8

P8411 「SvR-1」Problem 题解 12

P2869 [USACO07DEC] Gourmet Grazers G 题解 14

P5021 [NOIP 2018 提高组] 赛道修建题解 16

P8379 Two Operations 题解 19

P8378 [PFOI Round1] Two Sequences 题解

思路分析

把最终 `fa` 看成一片有根森林。对每个非根节点 v，直接执行一次 `merge(v, fa[v])`，可以得到一类合法操作序列。要保留最终的父子边，子节点必须先于父节点挂接。设共有 m 条边，h_v 表示以非根节点 v 为根的子树大小，则这类直接挂接序列的数量为：

$$\frac{m!}{\prod_{v\text{ 非根}} h_v}$$

不同事件树之间的操作也可以交错。若上式大于 2，仅直接挂接的序列就已超过要求。若上式等于 1，所有非平凡边只能构成一条链。路径压缩只会把父指针改成更高的祖先，不能把被跳过的边恢复；因此要保留链上的每条父子边，每个节点都必须在其父节点仍是根时直接挂到父节点，合并顺序唯一，不会恰有两种序列。

直接挂接序列恰有两种时，事件偏序中只有一对不可比事件：它们要么是两个单边分量中的唯一事件，要么是同一分叉点下的两个叶子。因此只有两种可能：两个单边事件，或一棵“扫帚形”事件树（链末端分出两个叶子）。

扫帚形事件树对应的最终 `fa` 中，分叉点 u 是非根节点；设 $p = fa[u]$，两个叶子为 x, y。除了直接挂接的两种顺序，还可以先执行 `merge(x,y)`、再执行 `merge(y,u)`，最后执行 `merge(x,p)`。最后一次 `find(x)` 会把 x, y 压到 u，再把 u 挂到 p，仍得到相同的最终 `fa`；若 p 上方还有链，再按原顺序完成那段链即可。交换 x, y 又给出一种不同序列。因此所有这种单棵事件树都至少对应四种序列，不能计入。

剩下两个单边事件。若它们指向同一个根，最终 `fa` 是三点星形；若指向不同根，则是两个大小为 2 的连通块。两次合并的先后可以交换，分别恰有两种序列。任何先把两个叶子互相挂接的序列都无法在第二次合并中把它们同时压成同根的两个孩子，因为 `find` 发生在第二次挂接之前；因此没有额外序列。

因此答案只来自一个三点星形，或两个单边连通块。

一个大小为 3 的连通块

选择一个最终根，再从其余 $n-1$ 个点中选择两个点，让它们都直接指向这个根。两次合并可以交换顺序，因此对应恰好两种序列。

这类数组有：

$$n\binom{n-1}{2}$$

个。

两个大小为 2 的连通块

选出四个点，将它们分成两对；每对中选择一个根，另一个点指向根。两个连通块的合并先后可以交换，因此恰好有两种序列。

对应数组数为：

$$\binom{n}{4} \cdot 3 \cdot 2^2 = 12\binom{n}{4}$$

合并两类计数：

$$n\binom{n-1}{2} + 12\binom{n}{4} = \frac{n(n-1)(n-2)^2}{2}$$

复杂度

每个测试用例只进行常数次运算，时间复杂度 $O(1)$，额外空间复杂度 $O(1)$。

代码

```
namespace TANGYIXIAO {
constexpr ll mod = 998244353, iv = 499122177;
ll n, a, b, c;
inline void solve(int Task_Id) {
    cin >> n;
    if (n < 3) {
        cout << 0 << "\n";
        return;
    }
    a = n % mod, b = (n - 1) % mod, c = (n - 2) % mod;
    cout << a * b % mod * c % mod * c % mod * iv % mod << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8380 Two Hypercubes 题解

思路分析

这里的等式是整数等式 $y^x = x^z$，只有最终答案对 $10^9 + 7$ 取模。

先单独处理 $x = 1$。此时必须有 $y = 1$，而 z 有 C 种取值，贡献为 C。

下面设 $x > 1$。令：

$$g = \gcd(x, z), \qquad a = \frac{x}{g}, \qquad b = \frac{z}{g}$$

于是 $\gcd(a, b) = 1$。对 $y^x = x^z$ 两边同时除以指数公因数 g，得到：

$$y^a = x^b$$

因为 a, b 互质，比较两边每个质因数的指数，可知存在唯一整数 $t \geq 2$，使得：

$$x = t^a, \qquad y = t^b$$

代回 $z = gb$，有：

$$z = \frac{t^a b}{a}$$

因此还需要 $a \mid t^a b$。又 $\gcd(a, b) = 1$，所以等价于 $a \mid t^a$，也就是 t 必须被 a 的每个不同质因子整除。记 $d = \mathrm{rad}(a)$，那么条件就是 $d \mid t$。

固定互质的 (a, b)，三个上界分别要求：

$$t^a \leq A, \qquad t^b \leq B, \qquad t^a \leq \left\lfloor \frac{aC}{b} \right\rfloor$$

令：

$$R = \min\left(\left\lfloor \sqrt[a]{A} \right\rfloor, \left\lfloor \sqrt[b]{B} \right\rfloor, \left\lfloor \sqrt[a]{\lfloor aC/b \rfloor} \right\rfloor \right)$$

那么 t 是 d 的倍数且 $2 \leq t \leq R$。若 $a = 1$，$d = 1$，要去掉 $t = 1$，贡献为 $\max(R - 1, 0)$；若 $a > 1$，则 $d \geq 2$，贡献为 $\lfloor R/d \rfloor$。

由于 $t \geq 2$ 且 $A, B \leq 10^{18}$，指数 a, b 都不超过 59。枚举所有 $1 \leq a, b \leq 59$ 且 $\gcd(a, b) = 1$ 的组合即可。

整数根用整数 Newton 迭代求初值，再用 128 位快速幂校正，避免浮点误差影响边界。

复杂度

每组数据枚举 $O(\log^2 V)$ 个指数对，其中 $V = 10^{18}$；整数根用 Newton 迭代和快速幂校正，时间复杂度为 $O(\log^2 V \log\log V)$。空间复杂度为 $O(\log^2 V)$（预处理所有指数对）。

代码

```
namespace TANGYIXIAO {
const int N = 6e1, M = 4e3 + 5;
const ll mod = 1e9 + 7;
ull A, B, C, ra[N], rb[N];
int pa[M], pb[M], pc;
ll ans;
inline u128 pw(ull x, int e, u128 lim) {
    u128 v = 1, b = x;
    for (; e;) {
        if (e & 1) {
            if (b > lim / v) {
                return lim + 1;
            }
            v *= b;
        }
        e >>= 1;
        if (e) {
            if (b > lim / b) {
                b = lim + 1;
            } else {
                b *= b;
            }
        }
    }
    return v;
}
inline bool chk(ull x, int e, u128 lim) {
    return pw(x, e, lim) <= lim;
}
inline ull rt(u128 x, int e) {
    if (x == 0) {
        return 0;
    }
    if (e == 1) {
        return (ull)x;
    }
    int k = 0;
    for (u128 v = x; v; v >>= 1) {
        k++;
    }
    ull r = 1ULL << ((k + e - 1) / e);
    for (;;) {
        u128 p = pw(r, e - 1, x);
```

```
        u128 q = p > x ? 0 : x / p;
        ull nr = (ull)(((u128)(e - 1) * r + q) / e);
        if (nr >= r) {
            break;
        }
        r = nr;
    }
    for (; chk(r + 1, e, x);) {
        r++;
    }
    for (; !chk(r, e, x);) {
        r--;
    }
    return r;
}
inline int rd(int x) {
    int r = 1;
    for (int p = 2; p * p <= x; p++) {
        if (x % p == 0) {
            r *= p;
            for (; x % p == 0;) {
                x /= p;
            }
        }
    }
    if (x > 1) {
        r *= x;
    }
    return r;
}
inline void ini() {
    if (pc) {
        return;
    }
    for (int a = 1; a < N; a++) {
        for (int b = 1; b < N; b++) {
            if (MATH::gcd(a, b) == 1) {
                pa[pc] = a, pb[pc] = b, pc++;
            }
        }
    }
    return;
}
inline void solve(int Task_Id) {
    cin >> A >> B >> C;
    ini();
    for (int e = 1; e < N; e++) {
        ra[e] = rt(A, e), rb[e] = rt(B, e);
    }
    ans = C % mod;
    for (int i = 0; i < pc; i++) {
```

```
        int a = pa[i], b = pb[i];
        int d = rd(a), mn = (a == 1 ? 2 : d);
        if (ra[a] < (ull)mn || rb[b] < (ull)mn) {
            continue;
        }
        u128 lim = (u128)C * a / b;
        ull r = min(rb[b], lim < (u128)A ? rt(lim, a) : ra[a]);
        ull cnt = a == 1 ? (r > 1 ? r - 1 : 0) : r / d;
        ans = (ans + cnt % mod) % mod;
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8412 「SvR-1」Hack Function! 题解

思路分析

令 $q = k - 1$。对正整数 A，反复进行 k 进制数位和后得到的数，与 A 模 q 同余，且最终结果在 1 到 q 之间。因此

$$f(A) = \begin{cases} A \bmod q, & A \not\equiv 0 \pmod q, \\ q, & A \equiv 0 \pmod q. \end{cases}$$

于是只需研究 $i^i \bmod q$，并把余数 0 映射为 q。

从 q 开始的周期

设 $P = q\varphi(q)$。当 $i \geq q$ 时，数列 $f(i^i)$ 以 P 为周期。

对任意素数幂 $s^e \mid q$，若 $s \mid i$，则 $i \geq q \geq e$，所以 $i^i \equiv 0 \pmod{s^e}$；$i + P$ 也满足这一点。若 $s \nmid i$，则 i 是模 s^e 的单位，而 P 是 $\varphi(s^e)$ 的倍数，因此

$$(i + P)^{i+P} \equiv i^i \pmod{s^e}.$$

由中国剩余定理，$i^i \bmod q$ 的周期整除 P。

可以用 $O(P)$ 时间生成这个周期。固定余数 $x = i \bmod q$，周期中的位置为 $i = q + x + tq$，因此

$$i^i \equiv x^{q+x}(x^q)^t \pmod q.$$

对每个 x 先计算初值和公比，再递推 $\varphi(q)$ 次即可。

构造区间

把区间按起点和终点分成三类。记周期段一个周期的和为 C。

- 若区间完全位于 $[1, q-1]$，直接枚举所有子区间。
- 若 $l \geq q$，区间完全位于周期段。把目标和写成 $p = cC + t$，其中 $0 \leq t < C$。先取 c 个完整周期，再在周期数组中寻找和为 t 的循环连续段；若 $t = 0$，直接输出 c 个周期。由于各项为正且 $t < C$，循环段长度小于 P，用双指针在线性时间内枚举所有起点即可。
- 若 $l < q$ 且右端点 $r \geq q$，区间先包含固定的前缀 $l \sim q-1$，随后接上从周期下标 0 开始的一段数列。减去前缀和后，周期部分可写成若干完整周期加一个短前缀；短前缀的和用周期前缀和数组二分查找。

以上三类覆盖所有可能的区间；若都找不到，就输出 $-1\ -1$。

题目要求的区间长度不超过 p，而 $p \leq 10^8$；构造出的端点也远小于 10^{18}，满足输出范围。

复杂度

设 $P = q\varphi(q)$。每组数据的时间复杂度为 $O(P + q^2)$，空间复杂度为 $O(P)$。

题目保证 $k \le 10^3$，所以 $q \le 999$ 且 $P \le 999 \cdot 998 = 997002$。周期数组按 `N = 1e6 + 5` 开辟；前缀数组长度 $m + 1$ 也在该范围内。瞬态数组按 `K = 1e3 + 5` 开辟。

代码

代码只列出模板末尾的 `namespace TANGYIXIAO` 部分。

```
namespace TANGYIXIAO {
using ll = long long;
const int N = 1e6 + 5, K = 1e3 + 5;
int a[K], b[N];
ll ps[N], sf[K];
int k, q, ph, m, st, len;
ll p, cs, c, t, nd, cc, rem, tot;

inline ll pw(ll x, int e, int y) {
    ll r = 1 % y;
    for (; e; e >>= 1) {
        if (e & 1) {
            r = r * x % y;
        }
        x = x * x % y;
    }
    return r;
}

inline int phi(int x) {
    int r = x;
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0) {
            r = r / d * (d - 1);
            for (; x % d == 0; x /= d) {
            }
        }
    }
    if (x > 1) {
        r = r / x * (x - 1);
    }
    return r;
}

inline bool chk(int *a, int n, ll v, int &st, int &len) {
    if (v <= 0) {
        return false;
    }
    int r = 0;
```

```
    ll sum = 0;
    for (int i = 0; i < n; i++) {
        for (; r < i + n && sum < v; r++) {
            sum += a[r % n];
        }
        if (sum == v) {
            st = i, len = r - i;
            return true;
        }
        if (r > i) {
            sum -= a[i];
        }
    }
    return false;
}

inline void out(ll l, ll r) {
    cout << l << "␣" << r << "\n";
    return;
}

inline void solve(int Task_Id) {
    cin >> k >> p;
    q = k - 1, ph = phi(q), m = q * ph;
    ps[0] = 0, sf[q] = 0;
    for (int i = 1; i < q; i++) {
        int x = (int)pw(i, i, q);
        a[i] = x ? x : q;
    }
    for (int x = 0; x < q; x++) {
        if (x == 0) {
            for (int t = 0; t < ph; t++) {
                b[t * q] = q;
            }
            continue;
        }
        ll v = pw(x, q + x, q), r = pw(x, q, q);
        for (int t = 0; t < ph; t++) {
            int j = x + t * q;
            b[j] = v ? (int)v : q;
            v = v * r % q;
        }
    }
    cs = 0;
    for (int i = 0; i < m; i++) {
        cs += b[i];
    }

    for (int l = 1; l < q; l++) {
        ll sum = 0;
        for (int r = l; r < q; r++) {
```

```
            sum += a[r];
            if (sum == p) {
                out(l, r);
                return;
            }
            if (sum > p) {
                break;
            }
        }
    }

    c = p / cs, t = p % cs;
    if (t == 0 && c > 0) {
        out(q, q + c * m - 1);
        return;
    }
    if (chk(b, m, t, st, len)) {
        out(q + st, q + st + c * m + len - 1);
        return;
    }

    for (int i = 0; i < m; i++) {
        ps[i + 1] = ps[i] + b[i];
    }
    for (int i = q - 1; i >= 1; i--) {
        sf[i] = sf[i + 1] + a[i];
    }
    for (int l = 1; l < q; l++) {
        if (p < sf[l]) {
            continue;
        }
        nd = p - sf[l], cc = nd / cs, rem = nd % cs;
        ll *it = lower_bound(ps, ps + m, rem);
        if (it == ps + m || *it != rem) {
            continue;
        }
        tot = cc * m + (it - ps);
        if (tot > 0) {
            out(l, q - 1 + tot);
            return;
        }
    }

    out(-1, -1);
    return;
}
} // namespace TANGYIXIAO
```

P8411 「SvR-1」Problem 题解

思路分析

做完题目后的高兴程度，是已经完成题目中 a_i 的最小值。每一步选出的题目会把这个最小值保持不变，或将它降低。

如果当前有多道题可以做，优先做其中 a_i 最大的题目。设当前可做题中 a_x 最大，而某个最优顺序的下一题是 y。在这个顺序里找到 x，把 x 移到最前面，并将原来从开头到 x 前面的题目整体后移一位。这样仍然合法：x 原本就可做；其余题目保持相对次序，且 x 不可能是它们的前置题，因为它们原本排在 x 前面。

设开始这段顺序前的高兴程度为 h。新顺序第一步的贡献是 $\min(h, a_x)$；由于 $a_x \geq a_y$，后续每一步的高兴程度依次与原顺序前 $j-1$ 步相同，其中 j 是 x 原来的位置。新顺序的第一项不小于原顺序第 j 项，因为原顺序第 j 项还受此前所有题目的最小值限制。因此交换后总和不会变小。重复应用这个交换，就得到每一步都选当前可做题中 a_i 最大者的最优方案。

用大根堆维护所有已经满足前置条件的题目。先做题目 1，随后每次取出堆顶并加入它的儿子。父亲编号小于儿子编号，所以按输入生成父亲关系后，用全局数组 `hd`、`nx` 保存树的邻接表；用 `hp` 保存堆，避免动态分配。

题目值由 32 位无符号数生成，答案至多为 $n(2^{32}-1)$，使用 `long long` 足够。

题目有 $n \leq 10^7$，全局数组按 `N = 1e7 + 5` 开辟，覆盖题目编号和堆中最多 $n-1$ 个待做节点。

复杂度

每道题至多进入和离开堆一次，时间复杂度 $O(n \log n)$，空间复杂度 $O(n)$。

代码

代码只列出模板末尾的 `namespace TANGYIXIAO` 部分。

```
namespace TANGYIXIAO {
const int N = 1e7 + 5;
uint32_t a[N], sd;
int n, hd[N], nx[N], hp[N], hs;
long long mn, ans;

inline uint32_t rnd() {
    sd ^= sd << 13;
    sd ^= sd >> 17;
    sd ^= sd << 5;
    return sd;
}

inline bool cmp(int x, int y) {
```

```
    return a[x] < a[y];
}

inline void solve(int Task_Id) {
    cin >> n >> sd;
    for (int i = 1; i <= n; i++) {
        a[i] = rnd(), hd[i] = -1;
    }
    for (int i = 2; i <= n; i++) {
        int f = rnd() % (i - 1) + 1;
        nx[i] = hd[f], hd[f] = i;
    }
    hs = 0;
    for (int v = hd[1]; v =/= -1; v = nx[v]) {
        hp[hs++] = v;
        push_heap(hp, hp + hs, cmp);
    }
    mn = a[1], ans = mn;
    for (int i = 1; i < n; i++) {
        pop_heap(hp, hp + hs, cmp);
        int u = hp[--hs];
        mn = min(mn, (long long)a[u]);
        ans += mn;
        for (int v = hd[u]; v =/= -1; v = nx[v]) {
            hp[hs++] = v;
            push_heap(hp, hp + hs, cmp);
        }
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P2869 [USACO07DEC] Gourmet Grazers G 题解

思路分析

每份牧草只能分给一头奶牛。奶牛同时提出价格下限和口感下限，因此直接按价格或口感排序都不能马上确定分配。先按口感从高到低处理奶牛：当处理当前奶牛时，所有口感已经达标的牧草都加入候选集合，之后的奶牛口感要求只会更低。

候选牧草中，价格低于当前奶牛要求的无法使用；在剩下的牧草里，选择价格最低的一份。

这样选择不会使答案变差。考虑一个最优方案中当前奶牛使用牧草 g，而贪心选择了另一份更便宜且口感达标的牧草 h。如果 h 原本分给了后面某头奶牛，就把 g 改分给那头奶牛：后面奶牛的口感要求不高于当前奶牛，故 g 仍然达标，且总价格不变或更低。若 h 没有被使用，直接用 h 替换 g 也不会增加花费。因此每一步都可以安全地取当前可用的最低价牧草。

用一个按价格维护的 `multiset` 保存候选牧草。若其中不存在价格不低于当前要求的牧草，则无法完成所有分配，输出 -1。

复杂度

排序和集合操作的总复杂度为 $O((n+m)\log(n+m))$，空间复杂度为 $O(n+m)$。

代码

```
namespace TANGYIXIAO {
const int N = 1e5 + 5;
int n, m, j;
ll ans;
multiset<ll> s;
struct C {
    ll x, y;
} a[N];
struct G {
    ll x, y;
} b[N];
inline bool cc(const C &x, const C &y) { return x.y > y.y; }
inline bool cg(const G &x, const G &y) { return x.y > y.y; }
inline void solve(int Task_Id) {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].x >> a[i].y;
    }
    for (int i = 1; i <= m; i++) {
        cin >> b[i].x >> b[i].y;
    }
    sort(a + 1, a + n + 1, cc);
    sort(b + 1, b + m + 1, cg);
```

```
    s.clear(), ans = 0, j = 1;
    for (int i = 1; i <= n; i++) {
        for (; j <= m && b[j].y >= a[i].y; j++) {
            s.insert(b[j].x);
        }
        auto it = s.lower_bound(a[i].x);
        if (it == s.end()) {
            cout << -1 << "\n";
            return;
        }
        ans += *it;
        s.erase(it);
    }
    cout << ans << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P5021 [NOIP 2018 提高组] 赛道修建题解

思路分析

答案具有单调性。给定一个长度 L，如果能修出至少 m 条互不共用道路、长度都不小于 L 的赛道，那么所有小于 L 的长度也都可行。因此二分答案，并设计一个树形贪心判断可行性。

把树以 1 为根。处理完一个节点的子树后，向父亲最多保留一条尚未完成的链；子树中已经组成完整赛道的道路不再参与之后的处理。

实现用前向星存树，并预先生成从根出发的遍历序。判定时按遍历序的逆序处理节点，这样不用递归遍历深度可能达到 n 的树。

对于节点 u，每个孩子 v 会提供一条长度为

$$x_v = f_v + w(u, v)$$

的链，其中 f_v 是孩子向上保留的长度。

如果 $x_v \geq L$，这条链本身就能成为一条赛道。其余链都短于 L，需要在 u 处两两配对，使长度和至少为 L。配对时从最短的链 x 开始，找仍未使用且长度不小于 $L - x$ 的最短链。若找不到，x 不可能与任何剩余链组成赛道，只能作为向上保留链的候选；若找到，就完成一条赛道。

这个贪心既能得到最多的配对数，也会尽量把较长的链留给后续。若最短链 x 与最短可配链 y 没有被配在一起，可以交换原配对中的端点：因为其他可配对象不比 y 短，交换后仍满足长度和要求。若 x 根本找不到搭档，它和所有未处理链都配不成赛道，跳过它不会减少答案。最终向父亲传递所有未配链中的最大值。

遍历完根节点后，若已经完成至少 m 条赛道，则长度 L 可行。二分上界取全树边权和除以 m，因为 m 条边不相交赛道的总长度不可能超过所有边权之和。

复杂度

一次可行性判断中，每条边至多进入、离开一次有序集合，复杂度为 $O(n \log n)$。二分后总复杂度为 $O(n \log n \log W)$，其中 W 为所有边权之和；空间复杂度为 $O(n)$。

代码

```
namespace TANGYIXIAO {
const int N = 5e4 + 5, M = 1e5 + 5;
struct E {
    int v, ne;
    ll w;
};
int n, m, h[N], fa[N], ord[N], k, ed, u, v, cnt;
E e[M];
```

```
ll f[N], pw[N], sum, l, r, mid, w, res;
multiset<ll> s;
inline void add(int u, int v, ll w) { e[++ed] = {v, h[u], w}, h[u] = ed; }
inline bool chk(ll lim) {
    cnt = 0;
    for (int i = n; i; i--) {
        int u = ord[i];
        s.clear();
        for (int j = h[u]; j; j = e[j].ne) {
            int v = e[j].v;
            if (fa[v] != u) {
                continue;
            }
            ll x = f[v] + pw[v];
            if (x >= lim) {
                cnt++;
                if (cnt >= m) {
                    return true;
                }
            } else {
                s.insert(x);
            }
        }
        res = 0;
        for (; !s.empty();) {
            auto it = s.begin();
            ll x = *it;
            s.erase(it);
            auto jt = s.lower_bound(lim - x);
            if (jt == s.end()) {
                res = max(res, x);
            } else {
                s.erase(jt);
                cnt++;
                if (cnt >= m) {
                    return true;
                }
            }
        }
        f[u] = res;
    }
    return cnt >= m;
}
inline void solve(int Task_Id) {
    cin >> n >> m;
    for (int i = 1; i < n; i++) {
        cin >> u >> v >> w;
        add(u, v, w);
        add(v, u, w);
        sum += w;
    }
```

```
    k = 1;
    ord[1] = 1;
    for (int i = 1; i <= k; i++) {
        int u = ord[i];
        for (int j = h[u]; j; j = e[j].ne) {
            int v = e[j].v;
            if (v == fa[u]) {
                continue;
            }
            fa[v] = u;
            pw[v] = e[j].w;
            ord[++k] = v;
        }
    }
    l = 0, r = sum / m + 1;
    for (; l + 1 < r;) {
        mid = (l + r) / 2;
        if (chk(mid)) {
            l = mid;
        } else {
            r = mid;
        }
    }
    cout << l << "\n";
    return;
}
} // namespace TANGYIXIAO
```

P8379 Two Operations 题解

思路分析

先考虑一次 Query 中，最后哪一组作为合并目标。假设最后保留的是组 i，这组当前人数为 s，糖果总数为 c_i，组内最多糖果数为 m_i。所有组的糖果总数记为 C，学生总数为 n。

可以先把其余所有组互相合并，最后再把它们整体合并进组 i。最后这一步中，其余学生都会被清零，再各自得到：

$$\frac{C-c_i}{n}$$

颗糖。原来在组 i 的学生则都增加同样的数，因此最后最多糖果数是：

$$m_i+\frac{C-c_i}{n}=\frac{C+(nm_i-c_i)}{n}.$$

这个值也是以组 i 为最终目标时的下界。因为在任何合并顺序中，组 i 原有的学生都不会作为合并源被清零；其余组的糖果总量最终都要并入这条目标组链。每次并入时，分摊的分母至多为 n，所以这些学生累计增加的糖果数至少为 $(C-c_i)/n$。把其余组先合成一组再整体并入，正好达到这个下界。

因此答案只需在所有当前组中取：

$$\min_i\left(m_i+\frac{C-c_i}{n}\right)=\frac{C+\min_i(nm_i-c_i)}{n}.$$

总糖果数 C 和总人数 n 在操作中不变。每组维护糖果总数 c_i、组内最大值 m_i，并用有序集合维护键值 nm_i-c_i。Change X Y 时合并两组的总数与最大值，再更新集合；Query 时取集合最小键即可。最后乘以 n 在模 10^9+7 下的逆元。

复杂度

初始化为 $O(n+k\log k)$，每次 Change 为 $O(\log k)$，每次 Query 为 $O(1)$；空间复杂度为 $O(n+k)$。

代码

```
namespace TANGYIXIAO {
const int N = 1e6 + 5, M = 1e1;
const long long MOD = 1e9 + 7;
int n, m, k, g[N], x, y;
long long s[N], mx[N], tot, iv, v;
char op[M];
set<pair<long long, int>> st;
inline long long pw(long long a, long long b) {
```

```
    long long r = 1;
    for (; b; b >>= 1, a = a * a % MOD) {
        if (b & 1) {
            r = r * a % MOD;
        }
    }
    return r;
}
inline long long get(int x) { return 1LL * n * mx[x] - s[x]; }
inline void solve(int Task_Id) {
    cin >> n >> m >> k;
    for (int i = 1; i <= n; i++) {
        cin >> g[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> v;
        s[g[i]] += v, mx[g[i]] = max(mx[g[i]], v), tot += v;
    }
    for (int i = 1; i <= k; i++) {
        st.emplace(get(i), i);
    }
    iv = pw(n, MOD - 2);
    for (int i = 1; i <= m; i++) {
        cin >> op;
        if (op[0] == 'Q') {
            cout << (tot + st.begin()->first) % MOD * iv % MOD << "\n";
        } else {
            cin >> x >> y;
            if (x == y) {
                continue;
            }
            st.erase({get(x), x}), st.erase({get(y), y});
            s[y] += s[x], mx[y] = max(mx[y], mx[x]);
            s[x] = mx[x] = 0;
            st.emplace(get(y), y);
        }
    }
    return;
}
} // namespace TANGYIXIAO
```